Especificación de Requisitos del Software (ERS)

Tienda online Level-Up Gamer

Asignatura: DSY1104 Desarrollo Fullstack II · Evaluación Parcial 2 · Duoc UC

Integrantes: Matias Korner, Patricicio Segovia, Leonardo Bravo

Control de versiones

Versión	Etapa	Descripción
1.0	Evaluación Parcial 1	Tienda en HTML, CSS, JavaScript.
2.0	Evaluación Parcial 2	Migración a React, componentes reutilizables, archivo de datos con CRUD, nuevas vistas y pruebas unitarias.

1. Introducción

Este documento especifica los requisitos de la versión 2.1 de Level-Up Gamer, una tienda online para gamers en Chile, desarrollada como frontend con React. Cubre la tienda pública, una vista de administración inicial, una fuente de datos simulada con CRUD y las pruebas unitarias. Quedan fuera el backend real y los pagos reales, y todos los requisitos están sujetos a validación del docente.

La versión 1.0 (Evaluación 1) fue la tienda en HTML, CSS y JavaScript. La versión 2.0 (Evaluación 2) agregó la migración a React, los componentes, los datos con CRUD y las pruebas unitarias. La versión 2.1 incorpora tres componentes reutilizables nuevos (`FiltroCategorias`, `CampoFormulario` y `BarraBusqueda`), amplía las pruebas unitarias y agrega la medición de cobertura de código.

1.1 Definiciones y siglas

Término	Significado
ERS	Especificación de Requisitos del Software
SPA	Aplicación de página única (Single Page Application)
Componente	Pieza reutilizable de interfaz en React, escrita como función
Props	Datos y funciones que un componente recibe desde su componente padre
Estado	Memoria interna de un componente (`useState`) para valores que cambian
Componente controlado	Campo cuyo valor lo entrega el componente padre por props; el campo no guarda una copia propia
CRUD	Crear, Leer, Actualizar y Eliminar
Vitest	Herramienta de pruebas unitarias y de integración rápida para aplicaciones basadas en Vite y React
Mock	Función o módulo simulado que reemplaza a una dependencia real durante una prueba
Cobertura	Porcentaje del código que ejecutan las pruebas
Puntos LevelUp	Puntos del programa de fidelización de la tienda

2. Descripción general

La aplicación es una SPA (aplicación de página única) hecha con React y Vite, con navegación mediante React Router y diseño responsivo con Bootstrap 5. Los datos se guardan en `localStorage` a través de `data/productos.js` (productos con CRUD), lo que simula una base de datos.

2.1 Funciones principales

- Registro e inicio de sesión de usuarios, con beneficio de descuento para correos Duoc.
- Exploración del catálogo con categorías, búsqueda, filtros y orden.
- Detalle de producto y carrito de compras con resumen de totales.
- Proceso de pago con datos del cliente y dirección de entrega, y confirmación de la compra.
- Perfil de usuario, historial de pedidos, referidos y puntos LevelUp.
- Blog y soporte técnico por WhatsApp.
- Vista de administración.

2.2 Tipos de usuario

Usuario	Descripción
Visitante	Navega el catálogo, el blog y el soporte sin iniciar sesión.
Cliente registrado	Compra, gestiona su perfil, ve sus pedidos y acumula puntos LevelUp.
Cliente Duoc	Cliente registrado con correo @duoc.cl o @profesor.duoc.cl, con 20% de descuento.
Administrador	Accede al panel de administración con todas las secciones; puede agregar, editar y eliminar productos.
Vendedor	Accede al panel de administración con permisos reducidos: ve productos y pedidos, pero no edita productos ni ve la sección de usuarios.

3. Requisitos funcionales

ID	Requisito	Estado	Módulo
RF-01	Registro solo para mayores de 18 años, con código de referido.	Implementado	RegistroUsuario, CampoFormulario
RF-02	Descuento de 20% para correos @duoc.cl y @profesor.duoc.cl.	Implementado	services/tienda.js
RF-03	Inicio de sesión y perfil de usuario.	Implementado	Login, Perfil
RF-04	Catálogo por categorías con búsqueda, filtros y orden.	Implementado	Catalogo, TarjetaProducto, BarraBusqueda
RF-05	Detalle de producto.	Implementado	DetalleProducto
RF-06	Carrito: agregar, cambiar cantidad, eliminar y ver totales.	Implementado	Carrito, ItemCarrito
RF-07	Pago con datos del cliente y dirección (región y comuna) validados.	Implementado	ProcesoPago
RF-08	Confirmación de compra e historial de pedidos.	Implementado	ConfirmacionPedido, ListaPedidos
RF-09	Puntos LevelUp, niveles y canje.	Implementado	SistemaPuntos
RF-10	Blog y soporte técnico por WhatsApp.	Implementado	Blog, Soporte

ID	Requisito	Estado	Módulo
RF-11	Fuente de datos de productos con CRUD.	Parcial	data/productos.js
RF-12	Panel de administración.	Parcial	MenuAdmin
RF-13	Vistas de ofertas, categorías y pago con error.	Pendiente	—
RF-14	Reseñas, mapa de eventos y recomendaciones.	Pendiente	—

4. Requisitos no funcionales

- Responsividad: diseño adaptable a móviles, tabletas y escritorios con Bootstrap 5.
- Identidad visual: fondo negro, acentos azul #1E90FF y verde #39FF14, y tipografías Roboto y Orbitron, según el caso.
- Mantenibilidad: cada componente tiene una sola responsabilidad, recibe datos por props y usa estado solo cuando es necesario. Los bloques que se repiten en las páginas se extraen a componentes reutilizables.
- Testabilidad: las pruebas se ejecutan con `npm test` y la cobertura se mide con `npm run coverage`.
- Compatibilidad: navegadores modernos; las pruebas se ejecutan en un navegador simulado (jsdom).
- Seguridad: `localStorage` es solo de demostración y no es seguro para producción.

5. Componentes de la aplicación

La aplicación tiene 27 componentes de React: App, 3 de estructura (Layout, Header y Footer), 7 transversales o reutilizables (ScrollToTop, ErrorBoundary, TarjetaProducto, ItemCarrito, FiltroCategorias, CampoFormulario y BarraBusqueda) y 16 páginas, una por cada vista. Todos son funciones, salvo ErrorBoundary, que es una clase porque React solo permite capturar errores de renderizado con una. Además hay 2 hooks personalizados que dan apoyo a los componentes.

5.1 Componentes de estructura y transversales

Componente	Qué hace	Props	Estado y efectos
App (src/App.jsx)	Define las rutas con `Routes` y `Route`. Agrupa 8 vistas dentro de Layout; las demás quedan fuera para no mostrar la barra de navegación.	Ninguna	`useLocation`: entrega la `key` de la ruta a ErrorBoundary.
Layout (components/layout)	Arma la estructura común: Header, el contenido de la vista (`Outlet`) y Footer.	Ninguna	Sin estado.
Header (components/layout)	Barra de navegación Bootstrap con logo, enlaces (Inicio, Catálogo, Blog, Soporte), carrito con contador y ícono de perfil. Sin sesión muestra Iniciar sesión y Registrarse, y un modal para invitados. En móvil el menú se pliega.	Ninguna	Estado `cantidadProductos` y hook `useSesion`. Efectos: actualiza el contador con los eventos `storage`, `focus`, `lug-carrito-actualizado` y `lug-cart-updated`; cierra el menú móvil al cambiar de ruta; limpia el fondo del modal al desmontarse.

Componente	Qué hace	Props	Estado y efectos
Footer (components/layout)	Pie de página con 4 columnas: logo y descripción, navegación, soporte y contacto.	Ninguna	Sin estado.
ScrollToTop (components)	Sube al inicio de la página cada vez que cambia la ruta o el parámetro de búsqueda. No dibuja nada (devuelve `null`).	Ninguna	`useEffect` con `pathname` y `search`.
ErrorBoundary (components)	Captura errores de renderizado y muestra PaginaError con código 500. Al navegar a otra ruta vuelve a intentar el renderizado normal.	`children`, `resetKey`	Estado `hayError`; métodos `getDerivedStateFromError`, `componentDidCatch` y `componentDidUpdate`.

5.2 Componentes reutilizables

Son componentes de presentación: no guardan datos, todo les llega por props.

Componente	Qué muestra	Props	Usado en
TarjetaProducto	Un producto del catálogo: imagen desde /img, nombre, precio con formato es-CL y enlace «Ver Detalle» a /detalle?id=.	`producto` (id, nombre, precio, imagen, alt, categorias)	Catalogo e Inicio
ItemCarrito	Un producto del carrito: imagen, nombre, variante, precio por cantidad, botón de eliminar y botones «-» y «+».	`item`, `onCambiarCantidad(id, cambio)`, `onEliminar(id)`	Carrito
FiltroCategorias	Fila de botones de categoría. El botón de la categoría activa se ve resaltado (sin la clase `boton-borde`) y marca `aria-pressed`. Al hacer clic llama a `onSeleccionar` con el valor de la categoría.	`categorias` (arreglo de {valor, texto}; por defecto vacío), `activa`, `onSeleccionar(valor)`	Inicio
CampoFormulario	Campo de formulario con un ícono a la izquierda. Por cada cambio llama a `onChange` con el evento; el atributo `name` toma el valor de `id` si no se indica.	`icono`, `id`, `name` (por defecto `id`), `tipo` (por defecto «text»), `placeholder`, `valor`, `onChange(evento)`	RegistroUsuario (nombre, correo y contraseñas)
BarraBusqueda	Campo de búsqueda con ícono de lupa. Avisa al padre el texto escrito (no el evento) con `onCambiar`. Incluye `aria-label` para accesibilidad.	`valor`, `onCambiar(texto)`, `placeholder` (por defecto «Buscar...»)	Catalogo
PaginaError	Pantalla de error con código, subtítulo y descripción, y un modal para reportar el problema con validación del formulario.	`codigo` (por defecto 404), `subtitulo`, `descripcion`	Rutas /404 y * y ErrorBoundary (500)

5.3 Páginas (una por vista)

Ruta	Vista
`/`	Inicio
`/catalogo`	Catálogo
`/detalle`	Detalle de producto
`/carrito`	Carrito de compras
`/pago`	Proceso de pago
`/confirmacion`	Confirmación de pedido
`/login`, `/registro`	Inicio de sesión y registro
`/perfil`	Perfil de usuario
`/pedidos`	Historial de pedidos
`/puntos`	Sistema de puntos LevelUp
`/blog`, `/articuloN1`	Blog y artículo
`/soporte`	Soporte técnico
`/admin`	Panel de administración
`/404` y cualquier otra ruta	Página de error

6. Fuente de datos de productos

`data/productos.js` actúa como base de datos simulada: la primera vez guarda los 13 productos iniciales en `localStorage` (clave `lug_productos`) y el catálogo lee desde ahí.

Función	Qué hace	Devuelve
`obtenerProductos()`	Lee todos los productos.	Arreglo de productos
`obtenerProductoPorId(id)`	Busca un producto por su id.	El producto, o `undefined`
`crearProducto(datos)`	Agrega un producto nuevo.	El producto creado, o `null` si falta el id o ya existe
`actualizarProducto(id, cambios)`	Cambia los campos indicados; el id no se puede modificar.	El producto actualizado, o `null` si no existe
`eliminarProducto(id)`	Borra un producto.	`true` si lo borró, `false` si no existía
`restablecerProductos()`	Vuelve a los datos iniciales.	Arreglo de productos

Nota: el panel de administración (MenuAdmin) mantiene su propia lista de productos en el estado del componente y aún no utiliza estas funciones; por eso RF-11 y RF-12 figuran como parciales.

7. Pruebas unitarias

El proyecto tiene 110 pruebas unitarias en 10 archivos, escritas con Vitest y Testing Library, que verifican componentes, páginas y la lógica de datos.

7.1 Tipos de prueba aplicados

Tipo	Qué verifica	Ejemplo
Renderizado	Que el componente muestre los datos recibidos.	ItemCarrito: nombre, variante y cantidad
Renderizado condicional	Que se muestren u oculten elementos según el caso.	Carrito vacío frente a Carrito con productos; Header con y sin sesión
Props	Que se usen las propiedades recibidas, incluidos los valores por defecto.	TarjetaProducto: nombre, precio, imagen y enlace; CampoFormulario: `tipo` y `name` por defecto
Eventos	Que un clic o un cambio ejecute la acción esperada.	ItemCarrito llama a `onCambiarCantidad(id, 1)`; BarraBusqueda llama a `onCambiar(texto)`
Estado y persistencia	Que el cambio se vea en pantalla y se guarde en `localStorage`.	Carrito: sumar una unidad y recalcular el total
Componentes controlados	Que el valor mostrado lo decida el padre y se actualice cuando cambian las props.	BarraBusqueda y FiltroCategorias con `rerender` y con un padre que tiene estado
Validación de formularios	Que se muestren los errores y se bloquee el envío.	Login: contraseña incorrecta, campos vacíos y dominio no permitido
Lógica de negocio	Que se apliquen las reglas de la tienda.	Carrito: 20% de descuento con correo @duoc.cl
Lógica de datos	Que el CRUD cumpla sus reglas.	productos.js: crear, actualizar y eliminar
Navegación	Que un enlace lleve a la vista correcta y que el enlace activo se marque.	TarjetaProducto: clic en «Ver Detalle»; Header: enlace activo
Manipulación del DOM	Que el componente modifique elementos fuera de su propio árbol y los limpie al salir.	Login: clase del `<body>` y título; Header: menú móvil y fondo del modal
Eventos globales	Que el componente reaccione a eventos de `window`.	Header: `lug-sesion-actualizada` y `lug-carrito-actualizado`
Mocks y espías	Que se llamen las dependencias con los datos correctos.	Catalogo con un módulo de datos simulado; Carrito con `toast` espiado; Login con `alert` simulado
Pruebas parametrizadas	Que una misma plantilla cubra varios casos.	Catalogo: los 4 criterios de orden; CampoFormulario: tipos «email» y «password»

7.2 Resumen por archivo

Archivo	Qué prueba	Pruebas
ItemCarrito.spec.jsx	Componente ItemCarrito	6
TarjetaProducto.spec.jsx	Componente TarjetaProducto	7
Carrito.spec.jsx	Página Carrito	12
Login.spec.jsx	Página Login	12
Header.spec.jsx	Componente Header	14
Catalogo.spec.jsx	Página Catalogo	18
FiltroCategorias.spec.jsx	Componente FiltroCategorias	7
CampoFormulario.spec.jsx	Componente CampoFormulario	11
BarraBusqueda.spec.jsx	Componente BarraBusqueda	9
productos.spec.js	Fuente de datos productos.js	14
Total		**110**